Singly Linked List –
Auxiliary Search –
Space Complexity and Algorithm.

Program:

```
void auxiliarySearch(Node **head, int key, Node **ploc, Node **loc)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }

    if ((*head)->data == key)
    {
        *loc = *head;
        return;
    }

    *ploc = *head;
    *loc = (*head)->next;

    while (*loc != nullptr)
    {
        if ((*loc)->data == key)
        {
            return;
        }
        else
        {
            *ploc = *loc;
            *loc = (*loc)->next;
        }
    }
}
```

```
…..
        case 12:
        {
            Node *ploc = nullptr, *loc = nullptr;
            int key;
            cout << "Enter the element to search: ";
            cin >> key;
            auxiliarySearch(&head, key, &ploc, &loc);
            if (loc != nullptr)
            {
                cout << "Element " << key << " found." << endl;
                if (ploc != nullptr && loc != nullptr)
                {
                    cout << "Previous element is " << ploc->data << "." << endl;
                    cout << "Searched Element is " << loc->data << "." << endl;
                }
                else
                {
                    cout << loc->data << " is at the head of the list." << endl;

                    cout << "Previous Element is absent ,and ploc's value is
                    nullptr." << endl;
                }
            }
            else
            {
                cout << "Element " << key << " not found." << endl;
            }
            break;
        }
```

Space Complexity

1. *Input Space*:

A. Parameter – only view: –

The input space is determined by the parameters passed to the function. In this case, the function takes four parameters:

- ***Node ** head***: ***A pointer to the head of the linked list***.
- ***int key***: ***The key to search for in the linked list***.
- ***Node ** ploc***: ***A pointer to store the location of the previous node***.
- ***Node ** loc***: ***A pointer to store the location of the node containing the key***.

Since these parameters are pointers and an integer, they require a constant amount of space, i.e., O(1).

• Node **head 1 machine-word

• int key 1 machine-word

• Node **ploc 1 machine-word

• Node **loc 1 machine-word

Four fixed-size words → constant memory, independent of the list size.

This view only considers the memory for the parameters passed to the function. The parameters are `Node **head`, `int key`, `Node **ploc`, and `Node **loc`. These consist of three pointers-to-pointers and one integer, all of which have a **fixed, constant size**. Therefore, the input space complexity is **O(1)**.

B. Full – data (inclusive) view

Add everything reachable from the input pointers at the instant the call begins. *head may reference a singly linked list containing n nodes, each of size ≈ sizeof(Node). All other parameters are already counted above. Therefore the total input footprint is Θ(n).

This more common definition includes the entire data structure. If the linked list has **n nodes**, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

2. *Auxiliary Space*:

Auxiliary space refers to any additional space used by the function that is not part of the input space. In this function, there are no additional data structures or recursive calls that consume extra space. The function operates directly on the input pointers and does not allocate any new memory during its execution.

What the routine itself adds beyond its input:

• Locals created by the compiler: a handful of registers/temporaries.

• Loop re-uses the two output pointers (*ploc, *loc) to walk the list; no new variables are allocated.

• No dynamic memory allocations and no recursion.

Because the amount of additional memory never grows with n, the auxiliary-space complexity is constant.

• Local stack space: only the function's own activation record and a handful of compiler-generated temporaries → Θ(1).

• No dynamic-heap allocations.

• No recursion, so stack depth is constant. ⇒ Θ(1) = O(1)

Auxiliary Space Complexity: Θ(1)

Auxiliary space is the extra memory an algorithm uses, not counting the input. This function is a prime example of constant auxiliary space usage.

1. **No New Variables**: The function does **not** declare any of its own local variables (like a `current` pointer or a counter `i`). Instead, it directly uses the memory locations provided by the `ploc` and `loc` parameters to traverse the list.
2. **Iteration**: The `while` loop simply modifies the pointers at the addresses given by `ploc` and `loc`. This process uses no additional memory.

Since the function operates entirely using its input parameters without creating any new variables, the amount of extra memory used is constant (effectively zero, which is a form of O(1)).

Algorithm

$AUXSEARCH(START, KEY, PREVPTR, CURRPTR)$

1. $[LIST\ is\ Empty?]$

$If\ START = NULL, then:$

$Write:$ ``Error: List is empty.``

$Return\ False\ and\ EXIT.$

$[END\ OF\ IF\ STRUCTURE]$

2. $[Key\ is\ found\ in\ First\ Node?]$

$If\ INFO[START] = KEY, then:$

$Set\ CURRPTR := START \begin{bmatrix} CURRPTR\ , pointer\ will\ get\ assigned\ to\ address\ that\ START\ or \\ HEAD\ pointer\ points\ to\ i.e.\ address\ of\ First\ Node. \end{bmatrix}$

$Return\ False\ and\ EXIT$

$[END\ OF\ IF\ STRUCTURE]$

$[When\ KEY\ is\ not\ found\ in\ First\ Node]$

3. $Set\ PREVPTR := START \begin{bmatrix} CURRPTR\ , pointer\ will\ get\ assigned\ to\ address\ that\ START\ or \\ HEAD\ pointer\ points\ to\ i.e.\ address\ of\ First\ Node. \end{bmatrix}$

4. $Set\ CURRPTR := LINK[START] \begin{bmatrix} CURRPTR, pointer\ will\ get\ assigned\ to\ address\ which\ is\ stored\ in\ LINK\ of \\ 1st\ Node. \end{bmatrix}$

5. $Repeat\ while\ CURRPTR \neq NULL, then:$

$if\ INFO[CURRPTR] = KEY, then:$

$Return\ and\ Exit.$

$[If\ KEY\ is\ not\ found\ in\ the\ NODE\ pointed\ by\ CURRPTR\ pointer]$

$ELSE$

$Set\ PREVPTR := CURRPTR \begin{bmatrix} Now\ address\ hold\ by\ CURRPTR\ assigned\ to\ PREVPTR\ and\ PREVPTR \\ will\ now\ point\ to\ the\ address. \end{bmatrix}$

$Set\ CURRPTR := LINK[CURRPTR] \begin{bmatrix} Now\ address\ hold\ by\ LINK\ of\ NODE\ that\ CURRPTR\ is \\ pointing\ assigned\ to\ CURRPTR\ which\ means\ CURRPTR \\ will\ now\ point\ to\ NEXT\ NODE\ . \end{bmatrix}$

$[END\ OF\ IF\ STRUCTURE]$

$[END\ of\ LOOP]$